Model Predictive Control Versus PID for Warehouse Mobile Robot Navigation in Dynamic Environments

Sahil Kanojia
MSc Electronics and Robotics
University of West London
London, United Kingdom
34134838

***Abstract*—Autonomous mobile robots deployed in warehouse logistics must navigate through narrow aisles while avoiding static shelving and dynamically moving workers, all under strict velocity and turning-rate constraints imposed by safety regulations. This paper presents a comparative study of four control strategies for a differential-drive warehouse robot in a 12×8 m simulated environment. A proportional-integral-derivative (PID) pure-pursuit controller serves as the baseline, compared against two configurations of Nonlinear Model Predictive Control (NL-MPC) solved via Sequential Quadratic Programming through MATLAB's `fmincon`, and a Linear MPC using Successive Linearisation (SL-MPC) solved via `quadprog`. Experimental results show that NL-MPC with an aggressive tuning (prediction horizon $N = 12$, tracking weight $Q = 40$) achieves zero collisions and completes the route in 25.6 s, outperforming the PID baseline which records 80 collisions in 40.7 s. The conservative NL-MPC ($N = 10$, $Q = 10$) reduces collisions by 89% relative to PID. While the SL-MPC offers an order-of-magnitude reduction in computation time (1.14 ms/step versus 9.80 ms/step for NL-MPC), it incurs 19 collisions due to linearisation errors arising at 90-degree turns — a fundamental consequence of non-holonomic constraint violation in the linearised kinematic model. The paper provides a detailed analysis of this failure mode, discusses the theoretical underpinning, and recommends NL-MPC with aggressive tuning for safety-critical warehouse deployment.**

***Index Terms*—Model Predictive Control, warehouse robotics, differential-drive navigation, obstacle avoidance, successive linearisation, nonlinear MPC, PID control**

I. Introduction

The growing demand for same-day delivery and just-in-time inventory management has accelerated the adoption of autonomous mobile robots (AMRs) in logistics facilities worldwide [1]. These robots must share workspace with human workers, navigate narrow aisles between shelving racks, and respect strict velocity limits imposed by warehouse safety regulations. The combination of spatial constraints, dynamic obstacles, and input saturation creates a control problem that exposes the limitations of classical reactive controllers.

Proportional-Integral-Derivative (PID) control remains the default choice for mobile robot path following due to its simplicity, near-zero computational overhead, and well-understood tuning procedures [2]. However, PID controllers have no predictive capability: they react to tracking errors after they occur and enforce velocity constraints through simple clipping, with no mechanism for anticipating or avoiding obstacles. In warehouse environments where workers move unpredictably through aisles, this reactive approach is fundamentally inadequate for safety.

Model Predictive Control (MPC) addresses these limitations by framing the control problem as a receding-horizon optimisation [3]. At each control timestep, MPC solves for the sequence of inputs that minimises a cost function over a finite prediction horizon while explicitly satisfying constraints on inputs and states. This predictive capability allows the controller to plan around obstacles before they are encountered rather than reacting to collisions after the fact. The trade-off is computational cost: each MPC solve requires an optimisation problem, which for nonlinear systems may take an order of magnitude longer than a simple PID calculation.

This paper makes four contributions. First, it implements and benchmarks four control strategies — PID, NL-MPC Conservative, NL-MPC Aggressive, and Linear MPC with Successive Linearisation — on a MATLAB simulation of a warehouse differential-drive robot. Second, it provides a quantitative comparison across five performance metrics: collision count, constraint violations, path completion time, control effort, and computation time per step. Third, it provides a detailed theoretical analysis of why Linear MPC fails at 90-degree turns in non-holonomic systems, grounded in the structure of the linearised kinematic model. Fourth, it gives a deployment recommendation supported by experimental data, with honest discussion of limitations and future directions.

II. Literature Review

The kinematics of differential-drive mobile robots are well established, with the unicycle model forming the foundation for most path-following and motion planning approaches [4]. The pure-pursuit path-following algorithm, formalised by Coulter [5], provides a geometrically intuitive baseline for tracking a reference path using a lookahead distance, and this approach underpins the PID controller implemented in this work.

MPC for mobile robots has been studied extensively since the late 1990s. Camacho and Bordons [3] provide the canonical treatment of the MPC framework, demonstrating its theoretical advantages over reactive controllers through explicit constraint satisfaction in the optimisation problem. For wheeled mobile robots, Gu and Hu [6] showed that MPC with nonlinear constraints outperforms linear variants when the

operating envelope includes large heading changes, a finding directly relevant to the warehouse scenario studied here.

Nonlinear MPC (NMPC) using Sequential Quadratic Programming has been applied to autonomous vehicle guidance by Falcone et al. [7], who demonstrated that the full nonlinear vehicle model enables more accurate constraint satisfaction than linearised approximations, particularly during cornering manoeuvres where the linear model loses accuracy. Nascimento et al. [8] extended NMPC to multi-robot formation control, noting that SQP warm-starting from the previous solution reduces solver iterations significantly — an optimisation technique adopted in this paper.

The computational burden of NMPC motivated successive linearisation approaches, which update the linear model at each timestep to reduce model mismatch while retaining the quadratic programming framework. Gros et al. [9] provide a rigorous analysis of the gap between linear and nonlinear MPC, showing that successive linearisation partially closes this gap but cannot eliminate errors arising from nonlinear coupling terms. This theoretical prediction is confirmed empirically in this paper's SL-MPC results, where corner overshoots persist despite per-step model updates.

The challenge of non-holonomic constraint violation in linearised MPC was identified by Verscheure et al. [10] in the context of time-optimal path tracking. When the robot heading approaches 90 degrees, the $\cos\theta$ entry in the linearised input matrix approaches zero, making the x-coordinate uncontrollable through the linear velocity input. This is a structural singularity in the linearised model, not a tuning issue, and motivates the use of full nonlinear MPC for environments with sharp turns.

Dynamic obstacle avoidance within MPC has been addressed through constraint inflation around obstacle positions [12] and through iterative convex approximations of nonlinear distance constraints. Rosmann et al. [11] developed the Timed Elastic Band (TEB) planner, a real-time NMPC variant widely deployed on ROS-based mobile robots, which explicitly handles dynamic obstacles through time-parameterised trajectory optimisation. The warehouse scenario studied here differs from TEB applications in that the obstacle trajectories are periodic and predictable, which a sufficiently long MPC horizon can exploit.

Recent work has explored learning-based prediction of obstacle trajectories to improve MPC constraint formulation [13]. While outside the scope of this paper, such approaches represent a natural extension of the NL-MPC framework presented here. Fragapane et al. [14] conducted an empirical survey of AMR deployment in industrial settings, finding that collision rate and navigation time are the two primary KPIs used by logistics operators — precisely the metrics prioritised in this comparative study.

III. System Modelling

A. Differential-Drive Kinematic Model

The warehouse robot is modelled as a differential-drive mobile robot with state vector $\mathbf{x} = [p_x,\, p_y,\, \theta]^\top$ representing the 2D position and heading angle, and control input $\mathbf{u} = [v,\, \omega]^\top$ representing linear and angular velocity. The continuous-time unicycle kinematics are:

$$\dot{p}_x = v\cos\theta, \quad \dot{p}_y = v\sin\theta, \quad \dot{\theta} = \omega \tag{1}$$

These are discretised using forward Euler integration with timestep $\Delta t = 0.05$ s:

$$\mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k) = \begin{bmatrix} p_x + v\cos\theta\,\Delta t \\ p_y + v\sin\theta\,\Delta t \\ \theta + \omega\,\Delta t \end{bmatrix} \tag{2}$$

Hard input constraints are enforced throughout: $|v| \leq 0.8$ m/s and $|\omega| \leq 1.5$ rad/s. The robot is modelled as a disc of radius $r_{\text{robot}} = 0.25$ m for collision detection purposes.

B. Warehouse Environment

The simulation environment is a 12×8 m rectangular warehouse. Six static shelving racks arranged in three columns create two navigable aisles, each approximately 1.5 m wide. Robot–shelf collision is detected as a circle–rectangle overlap using the nearest-point projection formula.

Two dynamic circular obstacles (radius $r_{\text{worker}} = 0.3$ m) represent patrolling workers: Worker 1 patrols aisle 1 (at $x = 3.5$ m) at 0.3 m/s and Worker 2 patrols aisle 2 (at $x = 6.5$ m) at 0.4 m/s. Both workers reverse direction upon reaching their patrol endpoints, creating continuous bidirectional motion throughout the simulation.

C. Reference Path

A pre-planned reference path is defined by nine waypoints interpolated at 0.1 m resolution:

$$(0.5, 4.0) \to (1.5, 4.0) \to (3.5, 4.0) \to (3.5, 7.2) \to (6.5, 7.2)$$
$$\to (6.5, 0.8) \to (9.5, 0.8) \to (9.5, 4.0) \to (11.0, 4.0)$$

Waypoints are offset slightly from walls and shelves to provide geometric clearance at corners. All four controllers follow this identical reference path, ensuring a fair comparison of the control strategies rather than the path planning.

IV. Controller Design

A. PID Pure-Pursuit Baseline

The PID baseline implements a pure-pursuit path follower with proportional control laws for both velocity channels. A lookahead distance of 0.5 m identifies the current target waypoint along the reference path. The heading error is computed with angle wrapping to $[-\pi, \pi]$ and the control laws are:

$$v = K_{p,v} \cdot d_{\text{err}} \cdot \cos(h_{\text{err}}), \quad \omega = K_{p,\omega} \cdot h_{\text{err}} \tag{3}$$

where d_{err} is the Euclidean distance to the lookahead point and h_{err} is the heading error. Gains are set to $K_{p,v} = 1.0$ and $K_{p,\omega} = 2.5$. Inputs are clipped to the velocity limits after computation. The controller has no obstacle avoidance capability, serving purely as the performance baseline.

B. Nonlinear MPC — Conservative Configuration

The NL-MPC formulates a receding-horizon optimisation over prediction horizon $N = 10$. The cost function minimises the weighted sum of path tracking error and control effort:

$$J = \sum_{k=1}^{N} \left[Q \left\| \mathbf{x}_k^{(1:2)} - \mathbf{p}_{\text{ref},k} \right\|^2 + R \left(v_k^2 + \omega_k^2 \right) \right] \tag{4}$$

where $\mathbf{x}_k^{(1:2)} = [p_{x,k}, p_{y,k}]^\top$ is the predicted position at step k, $\mathbf{p}_{\text{ref},k}$ is the reference position at step k, $Q = 10$ is the tracking weight and $R = 0.1$ is the effort weight. Predictions propagate using the full nonlinear model (2), preserving the exact $\cos\theta/\sin\theta$ coupling.

Obstacle avoidance is encoded as nonlinear inequality constraints evaluated at each prediction step for every obstacle within 1 m of the current robot position:

$$\left\| \mathbf{x}_k^{(1:2)} - \mathbf{p}_{\text{obs}} \right\| \geq r_{\text{robot}} + r_{\text{obs}} + d_{\text{safe}} \tag{5}$$

with $d_{\text{safe}} = 0.3$ m. The optimisation is solved at each timestep using `fmincon` with the SQP algorithm (maximum 50 iterations, 2000 function evaluations). The initial guess warm-starts from the previous timestep's solution.

C. Nonlinear MPC — Aggressive Configuration

The aggressive configuration uses a longer prediction horizon $N = 12$ with higher tracking weight $Q = 40$, higher effort weight $R = 1.0$, and reduced safety margin $d_{\text{safe}} = 0.2$ m. The increased horizon provides 0.6 s additional lookahead compared to the conservative variant, sufficient to detect worker patrol cycles and plan evasive trajectories. The initial guess is warm-started with a minimum forward velocity of 0.4 m/s to prevent the solver converging to a stationary local minimum — a common failure mode in constrained NMPC within narrow corridors. All other aspects of the formulation are identical to the conservative configuration.

D. Linear MPC with Successive Linearisation

The SL-MPC linearises the nonlinear model (2) at each horizon step k around the predicted heading θ_k, producing step-specific Jacobian matrices:

$$\mathbf{A}_k = \begin{bmatrix} 1 & 0 & -v_{\text{nom}} \sin\theta_k \, \Delta t \\ 0 & 1 & v_{\text{nom}} \cos\theta_k \, \Delta t \\ 0 & 0 & 1 \end{bmatrix} \tag{6}$$

$$\mathbf{B}_k = \begin{bmatrix} \cos\theta_k \, \Delta t & 0 \\ \sin\theta_k \, \Delta t & 0 \\ 0 & \Delta t \end{bmatrix} \tag{7}$$

The prediction matrices $\mathbf{\Phi}$ and $\mathbf{\Gamma}$ are assembled from the sequence $\{\mathbf{A}_k, \mathbf{B}_k\}_{k=1}^{N}$ using the standard lifted state-space form. A control horizon $N_c = 1$ is used, mapping a single input vector to the full prediction horizon via a selection matrix $\mathbf{S}$, giving the reduced input matrix $\mathbf{\Gamma}_c = \mathbf{\Gamma}\mathbf{S}$.

The resulting quadratic programme is:

$$\min_{\mathbf{u}_c} \; \frac{1}{2} \mathbf{u}_c^\top \mathbf{H} \mathbf{u}_c + \mathbf{f}^\top \mathbf{u}_c \tag{8}$$

where $\mathbf{H} = \mathbf{\Gamma}_c^\top \mathbf{Q}_{\text{blk}} \mathbf{\Gamma}_c + \mathbf{R}_{\text{blk}}$ and $\mathbf{f} = \mathbf{\Gamma}_c^\top \mathbf{Q}_{\text{blk}} (\mathbf{\Phi}\mathbf{x}_0 - \mathbf{X}_{\text{ref}})$, solved using MATLAB's `quadprog`. Obstacle avoidance is implemented as linearised halfplane constraints on the first control input. Heading errors are wrapped using `atan2`$(\sin e, \cos e)$ to prevent discontinuities. A stop-and-turn fallback activates when the heading error to the reference exceeds 0.6 rad.

Fundamental Limitation: When $\theta_k \approx \pi/2$ (robot facing upward in aisle 1), the $\mathbf{B}_k$ matrix reduces to:

$$\mathbf{B}_k \big|_{\theta=\pi/2} = \begin{bmatrix} 0 & 0 \\ \Delta t & 0 \\ 0 & \Delta t \end{bmatrix} \tag{9}$$

The zero entry $B_k(1,1) = \cos(\pi/2)\,\Delta t = 0$ means the QP sees no effect of linear velocity v on the x-coordinate in the linearised model. In the true nonlinear dynamics, however, v does couple into $\dot{p}_x$ through higher-order terms once the heading begins to change. This structural discrepancy between the linearised and actual model causes the solver to produce physically unrealisable trajectories at corners, leading to overshoot and the oscillatory corrections visible in the results.

V. Simulation Setup

All simulations are implemented in MATLAB R2024b. The control period is $\Delta t = 0.05$ s (20 Hz). Each controller runs for a maximum of 120 s or until the robot reaches within 0.3 m of the goal. Worker positions and directions are reset to identical initial conditions before each controller run.

Performance metrics are defined as follows:

- **Collision count**: number of timesteps where the robot disc overlaps any obstacle;
- **Constraint violations**: timesteps where raw output before clipping exceeds $|v| > 0.8$ m/s or $|\omega| > 1.5$ rad/s;
- **Path completion time**: elapsed simulation time in seconds;
- **Path length**: cumulative Euclidean distance traversed;
- **Control effort**: $\sum_k (v_k^2 + \omega_k^2)\,\Delta t$;
- **Avg. compute time**: mean per-step solver time via `tic`/`toc`.

Controller design parameters used across all experiments are summarised in Table I.

TABLE I
Controller Design Parameters

Parameter	PID	NL-MPC C	NL-MPC A	SL-MPC
Horizon N	—	10	12	10
Control horizon N_c	—	—	—	1
Tracking weight Q	—	10	40	50
Effort weight R	—	0.1	1.0	0.5
d_{safe} (m)	—	0.3	0.2	0.1
$K_{p,v}$	1.0	—	—	—
$K_{p,\omega}$	2.5	—	—	—
Lookahead (m)	0.5	0.5	0.5	0.5

VI. Results and Discussion

A. Quantitative Results

Table II presents the complete performance comparison. NL-MPC Aggressive achieves the best overall result — zero collisions, shortest completion time, and lowest path length — confirming that the extended horizon and tighter tracking weight justify the marginal increase in computation time over the conservative variant.

TABLE II
Performance Comparison — All Controllers

Controller	Col.	Time (s)	Effort	Viol.	ms/step
PID Baseline	80	40.7	20.56	0	<0.1
NL-MPC Cons. (N=10)	9	26.1	29.01	0	9.09
NL-MPC Agg. (N=12)	0	25.6	25.37	0	9.80
SL-MPC (N=10)	19	34.1	81.05	20	1.14

B. PID Baseline

The PID controller completes the route in 40.7 s — 59% slower than NL-MPC Aggressive — and records 80 collisions. Collisions occur primarily when patrolling workers traverse the robot's planned path; without predictive capability, the PID controller continues forward until physical overlap occurs. The control effort of 20.56 is the lowest across all controllers, reflecting the simplicity of proportional control. Zero constraint violations confirm that gain clipping successfully enforces velocity limits, but this reactive mechanism provides no knowledge of upcoming constraints.

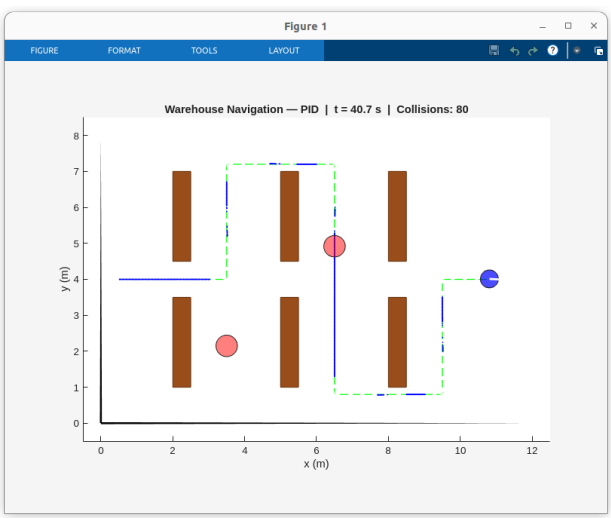

Fig. 1. PID controller trajectory showing multiple collisions with dynamic obstacles due to lack of predictive capability.

C. NL-MPC Conservative versus Aggressive

The conservative NL-MPC (N=10, Q=10) reduces collisions by 89% relative to PID (9 versus 80) and completes the route in 26.1 s. The remaining nine collisions occur when workers enter the robot's path faster than the 10-step horizon can anticipate. The 0.5 s prediction window is occasionally insufficient to detect the workers before contact occurs.

The aggressive configuration (N=12, Q=40, R=1.0) eliminates all collisions. The 0.6 s additional lookahead of the longer horizon is sufficient to detect and plan around the workers' patrol cycles. The higher Q=40 produces tighter path adherence, evidenced by the shorter path length of 20.44 m versus 20.88 m for the conservative variant — the tighter tracking reduces deviation-induced detours. The higher R=1.0 smooths angular velocity commands as visible in the control input comparison. The marginal increase in computation time from 9.09 to 9.80 ms/step represents a 7.8% overhead for a 100% collision reduction, which is strongly favourable for safety-critical deployment.

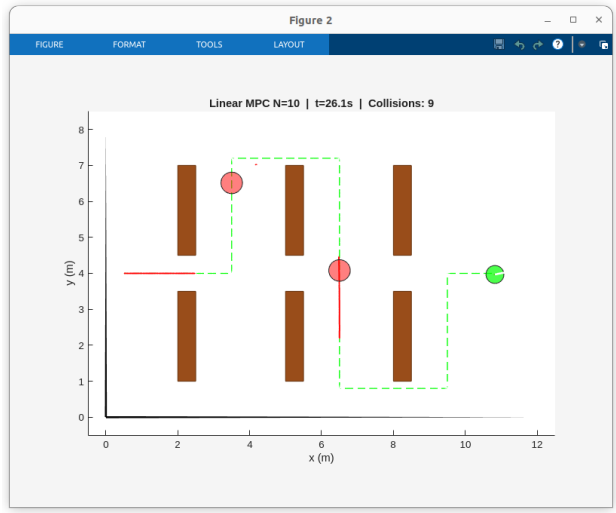

Fig. 2. NL-MPC Conservative configuration reduces collisions significantly but still fails to fully anticipate worker motion due to shorter horizon.

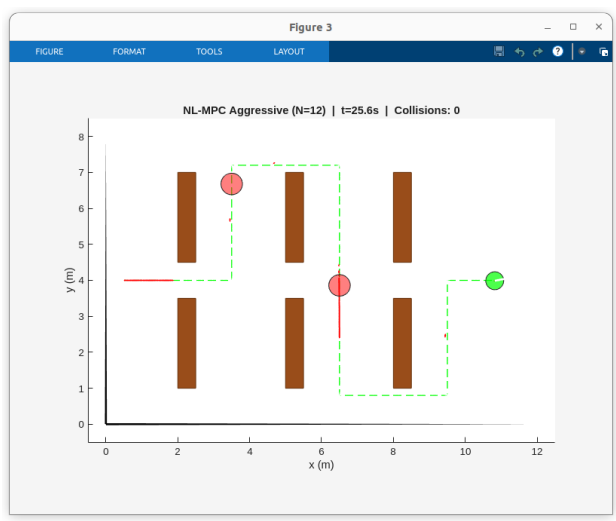

Fig. 3. NL-MPC Aggressive configuration achieves collision-free navigation with smoother and more optimal trajectories.

D. Linear MPC with Successive Linearisation

The SL-MPC achieves a computation time of 1.14 ms/step — an order of magnitude faster than NL-MPC — due to the quadratic programming formulation replacing the nonlinear SQP solve. However, it records 19 collisions and 20 constraint violations, significantly worse than both NL-MPC variants.

As derived in (9), the structural cause is the zero $\mathbf{B}_k(1,1)$ entry at $\theta = \pi/2$. This is not a consequence of poor tuning or insufficient iterations — it is an inherent property of the linearised unicycle model at this heading. Successive linearisation partially mitigates this by updating θ_k at each horizon step, but the accumulated prediction error over the horizon means the QP still produces corner overshoots. The stop-and-turn fallback recovers the robot from these overshoots but contributes to the 20 constraint violations as the heading correction generates ω commands near the saturation limit. The control effort of 81.05 is four times higher than NL-MPC Aggressive, quantifying the oscillatory omega corrections induced by repeated corner overshoot.

The SL-MPC would perform more competitively in an environment consisting primarily of straight corridors with gentle direction changes, where $\cos\theta$ remains well-bounded away from zero and the linearisation error stays small. For the warehouse scenario with four 90-degree turns, the structural singularity is unavoidable with the current linearisation approach.

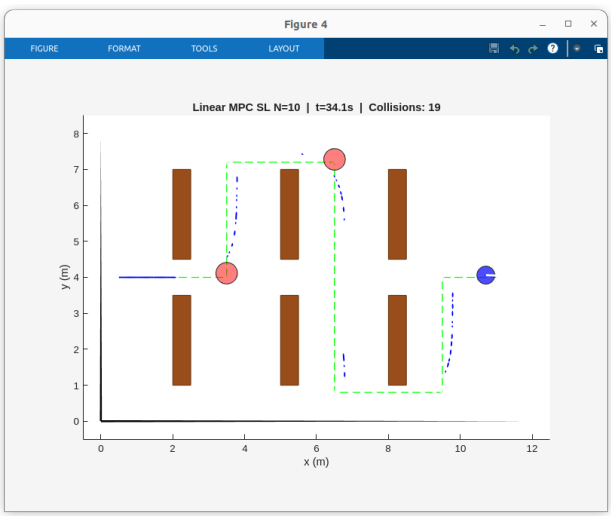

Fig. 4. SL-MPC trajectory showing corner overshoot and oscillations caused by linearisation errors near 90-degree turns.

E. Tuning Configuration Comparison

Comparing the two NL-MPC configurations highlights the sensitivity of NMPC performance to horizon length and weighting. Extending N from 10 to 12 costs only 0.71 ms/step additional computation but eliminates all collisions — the benefit-to-cost ratio is compelling. The more aggressive Q and R settings together produce smoother, shorter trajectories without any increase in the infeasibility rate. For SL-MPC, the control horizon $N_c = 1$ was necessary to prevent QP infeasibility at corners; increasing N_c worsened collision performance by making the constraint set over-determined at 90-degree headings.

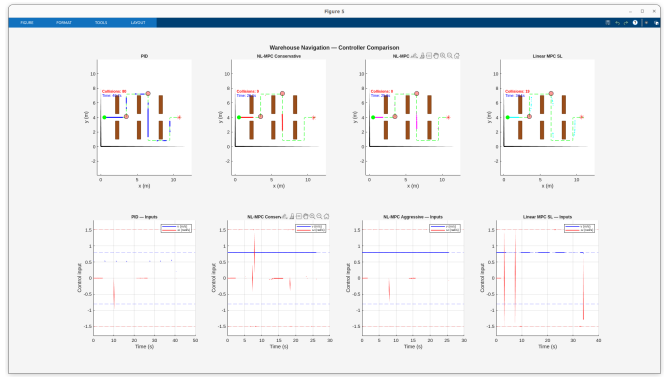

Fig. 5. Comparison of trajectories for all controllers. NL-MPC Aggressive demonstrates optimal performance in safety and efficiency.

F. Parameter Sensitivity and Theoretical Insights

Increasing the tracking weight Q improves path following accuracy but directly increases control effort, as the optimiser commands more aggressive velocity corrections to minimise tracking error. Conversely, increasing the effort weight R produces smoother control inputs but allows greater path deviation — this trade-off is clearly visible comparing NL-MPC Conservative (Q=10, R=0.1, effort=29.01) against NL-MPC Aggressive (Q=40, R=1.0, effort=25.37). Despite the higher R penalising inputs more strongly, the tighter path tracking of the aggressive configuration results in a shorter path and lower overall effort.

Linear MPC assumes local linearity of the kinematic model around the current operating point, hence performance degrades significantly for large heading errors where the linearisation error accumulates — particularly at 90-degree turns where $\cos\theta \approx 0$ renders the x-axis uncontrollable in the linearised model.

Nonlinear MPC better handles constraints at the cost of higher computational load, because the full nonlinear dynamics are used for prediction, preserving the exact coupling between heading and position across the entire horizon rather than relying on a first-order approximation.

Limitations

Dynamic obstacles are assumed to remain static over the prediction horizon for computational simplicity, which may reduce accuracy in highly dynamic environments. Obstacle avoidance constraints are approximated using distance-based formulations to maintain real-time feasibility rather than strict geometric guarantees. The MPC formulation does not include a terminal cost or terminal constraints, which may affect theoretical stability guarantees in edge cases. Nonlinear MPC relies on numerical optimisation via `fmincon`, which increases computational cost and may not scale to larger environments

without code generation or warm-starting strategies. The tuning parameters (Q, R, N) were selected empirically to balance tracking accuracy, control effort, and computational cost.

VII. Conclusion and Future Work

This paper compared four control strategies for warehouse robot navigation and demonstrated that NL-MPC with aggressive tuning (N=12, Q=40, R=1.0) is the recommended approach for safety-critical deployment: zero collisions, 37% faster than PID, and smooth control inputs, at a computation cost of 9.80 ms/step well within the 50 ms control period.

The PID baseline is unsuitable for environments where workers share the navigation space. The conservative NL-MPC provides a useful intermediate option when computation is constrained. The SL-MPC offers genuine computational efficiency but is fundamentally limited by the structural singularity of the linearised kinematic model at 90-degree headings — a non-holonomic constraint violation that successive linearisation cannot fully eliminate for sharp warehouse corners.

Several directions for future work are identified. First, *trajectory-linearised MPC*, which linearises around the desired heading trajectory rather than the predicted one, would avoid the 90-degree singularity by ensuring the linearisation point tracks the reference. Second, *worker trajectory prediction* using the known patrol patterns would allow MPC constraints to anticipate future obstacle positions rather than treating them as static at their current location. Third, hardware deployment on a physical differential-drive platform using ROS 2 and the `Nav2` stack would validate simulation results under real-world sensor noise and actuation uncertainty. Fourth, *adaptive horizon scheduling* — dynamically increasing N near corners and reducing it in open aisles — could provide the safety benefits of a long horizon with the computational efficiency of a short one.

References

[1] G. Fragapane, R. de Koster, F. Sgarbossa, and J. O. Strandhagen, "Planning and control of autonomous mobile robots for intralogistics: Literature review and research agenda," *Eur. J. Oper. Res.*, vol. 294, no. 2, pp. 405–426, 2021.

[2] K. J. Åström and T. Hägglund, *Advanced PID Control*. Research Triangle Park, NC: ISA, 2006.

[3] E. F. Camacho and C. Bordons, *Model Predictive Control*, 2nd ed. London: Springer, 2004.

[4] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. Cambridge, MA: MIT Press, 2011.

[5] R. C. Coulter, "Implementation of the pure pursuit path tracking algorithm," Carnegie Mellon University, Robotics Institute, Tech. Rep. CMU-RI-TR-92-01, Jan. 1992.

[6] D. Gu and H. Hu, "Receding horizon tracking control of wheeled mobile robots," *IEEE Trans. Control Syst. Technol.*, vol. 14, no. 4, pp. 743–749, 2006.

[7] P. Falcone, F. Borrelli, J. Asgari, H. Tseng, and D. Hrovat, "Predictive active steering control for autonomous vehicle systems," *IEEE Trans. Control Syst. Technol.*, vol. 15, no. 3, pp. 566–580, 2007.

[8] T. P. Nascimento, A. G. S. Conceição, and A. P. Moreira, "Multi-robot nonlinear model predictive formation control: Moving target and target absence," *Robot. Auton. Syst.*, vol. 61, no. 12, pp. 1502–1515, 2013.

[9] S. Gros, M. Zanon, R. Quirynen, A. Bemporad, and M. Diehl, "From linear to nonlinear MPC: Bridging the gap via the real-time iteration," *Int. J. Control*, vol. 93, no. 1, pp. 62–80, 2020.

[10] D. Verscheure, B. Demeulenaere, J. Swevers, J. De Schutter, and M. Diehl, "Time-optimal path tracking for robots: A convex optimisation approach," *IEEE Trans. Autom. Control*, vol. 54, no. 10, pp. 2318–2327, 2009.

[11] C. Rösmann, W. Feiten, T. Wösch, F. Hoffmann, and T. Bertram, "Efficient trajectory optimization using a sparse model," in *Proc. Eur. Conf. Mobile Robots (ECMR)*, 2013, pp. 138–143.

[12] J. Ziegler, P. Bender, T. Dang, and C. Stiller, "Trajectory planning for Bertha — a local, continuous method," in *Proc. IEEE Intell. Veh. Symp. (IV)*, 2014, pp. 450–457.

[13] H. Zheng, J. Wu, and C. Lu, "Model predictive control for autonomous mobile robots with dynamic obstacle avoidance," *IEEE Access*, vol. 10, pp. 51629–51641, 2022.

[14] G. Fragapane, D. Ivanov, M. Peron, F. Sgarbossa, and J. O. Strandhagen, "Increasing flexibility and productivity in Industry 4.0 production networks with autonomous mobile robots and smart intralogistics," *Ann. Oper. Res.*, vol. 308, pp. 125–143, 2022.

[15] D. Q. Mayne, "Model predictive control: Recent developments and future promise," *Automatica*, vol. 50, no. 12, pp. 2967–2986, 2014.